

Spring Boot Interview Questions for 2025 – Real Examples & Explanations

Get ready for your next Java interview with this ultimate guide to Spring Boot interview questions in 2025. Learn key concepts like auto-configuration, REST API creation, dependency injection, Actuator, security, and more.

If you are preparing for Java interview, chances are you will face questions about Spring Boot. It is a favorite for building modern, scalable and industry ready applications. Whether you are new to Spring Boot or brushing up for a interview, this article will definitely help you.

In this article, I have explained top spring boot interview questions in a simple and friendly way, making them easy to understand.

1. What is Spring Boot, and why is it used?

Spring Boot is an extension of the Spring framework designed to simplify building production-ready Java applications. It offers auto-configuration, embedded servers, and “starter” dependencies to reduce setup time and boilerplate code.

Why it's used:

- **Rapid development:** Auto-configuration and starters streamline setup, letting you focus on business logic.
- **Production-ready:** Built-in monitoring (via Actuator), logging, and metrics make it enterprise-friendly.
- **Microservices:** Ideal for building scalable, independent services.
- **Embedded servers:** No need for external servers like Tomcat; everything runs out of the box.

It's popular for web apps, APIs, and microservices due to its simplicity and robust ecosystem.

2. What are the key features of Spring Boot?

Spring Boot's key features make it a developer favorite:

- Auto-Configuration: Automatically sets up beans based on dependencies (e.g., database or web server configs).
- Starters: Pre-packaged dependencies for specific use cases (e.g., spring-boot-starter-web for REST APIs).
- Embedded Servers: Includes Tomcat, Jetty, or Undertow for instant deployment.
- Spring Boot Actuator: Provides endpoints like /health and /metrics for monitoring.
- Spring Boot CLI: Run apps or scripts from the command line.
- Production-ready tools: Built-in support for logging, profiles, and externalized configuration.
- Spring Initializr: A web tool to bootstrap projects with the right dependencies.

These features cut down development time and ensure scalability.

3. What is the @SpringBootApplication annotation?

@SpringBootApplication is a convenience annotation that combines three key annotations:

- @EnableAutoConfiguration: Enables Spring Boot's auto-configuration magic.
- @ComponentScan: Scans for Spring components (e.g., @Controller, @Service) in the package and sub-packages.
- @Configuration: Marks the class as a source of bean definitions for the Spring context.

This single annotation sets up your entire Spring Boot application, making it the entry point for most projects.

Example:

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

```
}  
}
```

4. What are Spring Boot Starters?

Starters are dependency descriptors that bundle related libraries for specific functionality, like web development, data access, or security. They simplify dependency management by ensuring compatible versions of libraries.

Examples:

- spring-boot-starter-web: For building REST APIs or MVC apps.
- spring-boot-starter-data-jpa: For database access with Hibernate.
- spring-boot-starter-security: For authentication and authorization.

Instead of manually adding individual libraries, you include a starter, and Spring Boot handles the rest.

Example:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

5. What is Spring Boot Auto-Configuration?

Auto-Configuration is Spring Boot's ability to automatically configure beans based on the dependencies in your classpath. For example, if you include spring-boot-starter-data-jpa, Spring Boot sets up a DataSource, JPA, and Hibernate without manual configuration.

How it works:

- Scans your classpath for libraries (e.g., H2, MySQL, or Tomcat).
- Applies sensible defaults (e.g., database connection settings).
- Allows customization via application.properties or application.yml.

This reduces boilerplate and speeds up development, but you can override defaults if needed.

Example:

```
# application.properties
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
```

```
spring.datasource.username=root
```

```
spring.datasource.password=password
```

6. How does Spring Boot differ from Spring Framework?

- Spring Framework: A powerful but complex framework requiring manual configuration (e.g., XML or Java config) for beans, servers, and dependencies. It's flexible but time-consuming.
- Spring Boot: Built on Spring, it simplifies setup with auto-configuration, starters, and embedded servers. It's designed for rapid development and production-ready apps.

Key differences:

- Spring Boot reduces boilerplate and setup time.
- Spring Boot includes embedded servers; Spring requires external servers.
- Spring Boot is opinionated with defaults; Spring offers more control.

7. What is the Spring Boot Actuator?

Spring Boot Actuator provides production-ready endpoints for monitoring and managing your application. These endpoints expose information like health status, metrics, environment details, and more.

Common endpoints:

- `/actuator/health`: Checks if the app and its dependencies (e.g., database) are up.
- `/actuator/metrics`: Provides performance metrics (e.g., memory usage).
- `/actuator/info`: Displays custom app info.

****How to enable:**** Add `spring-boot-starter-actuator` to your project and configure which endpoints to expose in `application.properties`.

Example:

```
# application.properties
```

```
management.endpoints.web.exposure.include=health,info,metrics
```

8. What is the purpose of application.properties or application.yml?

These files store configuration settings for your Spring Boot application, such as database connections, server ports, logging levels, or custom properties. application.yml is preferred for complex, hierarchical configs due to its readability.

Example use cases:

- Set server port: `server.port=8081`
- Configure database:
`spring.datasource.url=jdbc:mysql://localhost:3306/mydb`
- Enable Actuator endpoints:
`management.endpoints.web.exposure.include=*`

They externalize configuration, making it easy to tweak settings without changing code.

Example (application.yml):

```
server:  
  port: 8081  
spring:  
  datasource:  
    url: jdbc:mysql://localhost:3306/mydb  
    username: root  
    password: password
```

9. How do you create a REST API in Spring Boot?

To create a REST API, use `@RestController` with annotations like `@GetMapping`, `@PostMapping`, etc., to define endpoints. Spring Boot's `spring-boot-starter-web` provides all necessary dependencies.

Steps:

1. Add `spring-boot-starter-web` to your project.
2. Create a `@RestController` class with endpoint methods.
3. Use annotations to map HTTP requests to methods.

This sets up a RESTful service with minimal code, leveraging Spring Boot's embedded server.

Example:

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class MyController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello from Spring Boot!";
    }
}
```

10. What is Dependency Injection in Spring Boot?

Dependency Injection (DI) is a design pattern where dependencies are injected into a class rather than created inside it. Spring Boot uses annotations like `@Autowired` to manage DI.

Benefits:

- Reduces tight coupling between classes.
- Makes testing easier with mock dependencies.
- Centralizes bean management in the Spring context.

Example:

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@Service
public class MyService {
    public String getMessage() {
        return "Hello from Service!";
    }
}

@RestController
public class MyController {
    @Autowired
    private MyService myService;
    @GetMapping("/message")
```

```
public String getMessage() {  
    return myService.getMessage();  
}  
}
```

11. What is Spring Boot Starter Parent?

Spring Boot Starter Parent is a parent POM file in Maven that provides default configurations, dependency versions, and plugins for Spring Boot projects. It ensures consistent builds and simplifies pom.xml.

Key benefits:

- Predefined dependency versions (no need to specify versions for Spring libraries).
- Default plugins for building and packaging (e.g., executable JARs).
- Standard Java version and encoding settings.

Example (pom.xml):

```
<parent>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-parent</artifactId>  
    <version>3.3.2</version>  
</parent>
```

12. How do you connect a Spring Boot application to a database?

Use spring-boot-starter-data-jpa for database access with JPA/Hibernate. Configure the database in application.properties or application.yml.

Example:

```
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface UserRepository extends JpaRepository<User, Long> {  
}  
@Entity  
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String name;
```

```
// Getters and setters
}

spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update
```

13. What is Spring Boot DevTools?

Spring Boot DevTools enhances development by enabling features like automatic restarts when code changes and live reload in browsers.

Add this to your pom.xml, and your app restarts automatically on code changes :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
</dependency>
```

14. How do you handle exceptions in Spring Boot?

Use `@ControllerAdvice` and `@ExceptionHandler` to handle exceptions globally across your application. This ensures consistent error responses for REST APIs.

Key points:

- `@ControllerAdvice` defines a global exception handler.
- `@ExceptionHandler` specifies which exceptions to catch and how to respond.
- Return custom error messages or HTTP status codes (e.g., 400, 500).

This approach keeps error handling centralized and reusable.

Example:

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
```

```
@ControllerAdvice
```

```
public class GlobalExceptionHandler {
    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<String> handleIllegalArgumentException(IllegalArgumentException ex) {
        return new ResponseEntity<>("Error: " + ex.getMessage(), HttpStatus.BAD_REQUEST);
    }
}
```

15. What is the @EnableAutoConfiguration annotation?

@EnableAutoConfiguration enables Spring Boot's auto-configuration, which sets up beans based on classpath dependencies. It's part of @SpringBootApplication.

Example:

```
@SpringBootApplication // Includes @EnableAutoConfiguration
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

16. How do you implement security in Spring Boot?

Use spring-boot-starter-security to add authentication and authorization. Configure security rules using a SecurityFilterChain bean.

Key features:

- Secure endpoints with role-based access.
- Support for OAuth2, JWT, or form-based login.
- Customizable via SecurityFilterChain to define access rules.

This makes it easy to secure REST APIs or web apps.

Example:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
```

```

public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests()
            .requestMatchers("/public").permitAll()
            .anyRequest().authenticated()
            .and().formLogin();
        return http.build();
    }
}

```

17. What is the difference between @Controller and @RestController?

- @Controller: Marks a class as a web controller for MVC apps, typically returning view names (e.g., for Thymeleaf).
- @RestController: A specialized @Controller that returns data (e.g., JSON) directly, combining @Controller and @ResponseBody.

Example:

```

@RestController
public class MyRestController {
    @GetMapping("/api/hello")
    public String sayHello() {
        return "Hello from REST!";
    }
}

```

18. How do you create a Spring Boot executable JAR?

Spring Boot's Maven or Gradle plugin packages your app into an executable JAR with all dependencies and an embedded server.

Steps:

1. Include the Spring Boot plugin in pom.xml or build.gradle.
2. Run mvn package or gradle build.
3. Execute the JAR with java -jar target/myapp.jar.

This creates a single, deployable artifact for easy distribution.

19. What is Spring Boot Actuator's /health endpoint?

The /health endpoint, part of Spring Boot Actuator, shows the application's health status, including dependencies like databases or external services.

Details:

- Returns {"status":"UP"} if the app is healthy.
- Can include details (e.g., database status) with management.endpoint.health.show-details=always.
- Enabled with spring-boot-starter-actuator and exposed via properties.

It's critical for monitoring in production environments.

Example:

```
management.endpoints.web.exposure.include=health
```

20. How do you implement caching in Spring Boot?

Use spring-boot-starter-cache with annotations like @EnableCaching and @Cacheable to cache expensive operations.

Steps:

1. Add spring-boot-starter-cache.
2. Enable caching with @EnableCaching.
3. Use @Cacheable on methods to cache their results.

Example:

```
import org.springframework.cache.annotation.Cacheable;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.stereotype.Service;
```

```
@EnableCaching
@Service
public class MyService {
    @Cacheable("data")
    public String getData(String key) {
        // Simulate slow operation
        return "Data for " + key;
    }
}
```

Conclusion

Spring Boot is famous for building modern Java applications, and these 20 questions cover the core topics like auto-configuration, REST APIs, caching, security, and more. If you understand these well, you'll feel much more confident and definitely stand out in interviews.